

Tarea 1

Algoritmo de Prim y Análisis Amortizado

Profesores: Benjamín Bustos y Gonzalo Navarro

Auxiliares: Cristóbal Puentes y Gabriel Tapia

1. Indicaciones administrativas

- Fecha de entrega: Lunes 28 de septiembre hasta las 23.59. Atrasos hasta el 4 de octubre a las 23:59, con descuento (ver reglas del curso).
- Grupos de a lo más 3 personas **de la misma sección**.
- Lenguaje a utilizar: C, C++ o Java.

2. Contexto

Dado un grafo conexo, no dirigido y con pesos $G = (V, E)$, donde cada arista $e \in E$ tiene un peso $w(e) > 0$ asociado, su **árbol cobertor mínimo** (*minimum spanning tree* en inglés, o *MST*) es un subgrafo $G' = (V, E')$ con $E' \subseteq E$ que conecta todos los vértices de V y que minimiza $\sum_{e \in E'} w(e)$. Es decir, conecta todos los vértices usando el menor peso total posible.

Obs: Notar que, al ser G' conexo, acíclico y cobertor, se cumple $|E'| = |V| - 1$.

El objetivo principal de esta tarea es evidenciar empíricamente cómo el costo **amortizado** de las operaciones de una cola de prioridad impacta en la complejidad global de un algoritmo que la utiliza intensivamente para resolver su cuello de botella. Para ello implementaremos el algoritmo de Prim sobre dos colas de prioridad distintas: Colas binomiales y colas de Fibonacci.

3. Conceptos preliminares

3.1. Operaciones requeridas

El algoritmo de Prim necesita una estructura Q que soporte, como mínimo:

- Almacenar pares de la forma (costo, nodo), ordenados por costo.
- extractMin**(Q): obtener y eliminar el par de menor costo.
- decreaseKey**(Q, v, c): acceder al par que representa al nodo v y reducir su costo a c .

Notar que, a diferencia de otras aplicaciones, aquí nunca se necesita **aumentar** una clave: solo reducirla. Esto es justamente lo que ambas estructuras aprovechan.

3.2. decreaseKey en una cola binomial

Sea x el nodo cuya clave se quiere reducir. Como cada árbol de la cola es un B_k y cumple orden de *heap*, entonces basta con lo siguiente: se actualiza la clave y, mientras x tenga padre y su clave sea menor que la de éste, se intercambian. En pseudocódigo:

DECREASEKEY-BINOMIAL(Q, x, c)

```
1  $x.key \leftarrow c$ 
2  $y \leftarrow x.parent$ 
3 while  $y \neq \text{null}$  and  $x.key < y.key$ :
4    $x.content \leftrightarrow y.content$ 
5    $x \leftarrow y$ 
6    $y \leftarrow x.parent$ 
7 Actualizar el mínimo de  $Q$  si corresponde
```

Como la altura de B_k es $k + 1 \leq \lceil \log_2 n \rceil + 1$, el costo es $O(\log n)$ **en el peor caso**.

Importante: Tras el intercambio de la línea 4, cada uno de los dos nodos representa a un vértice distinto del que representaba antes; por lo mismo, los punteros que el algoritmo mantiene hacia el par de cada vértice quedan apuntando al nodo equivocado, y hay que actualizarlos junto con el intercambio. Si prefieren no tocarlos, deberán reconectar los nodos modificando los punteros del árbol. Ambas alternativas son válidas, pero deben justificar cuál eligieron en su informe.

3.3. decreaseKey en una cola de Fibonacci

En una cola de Fibonacci no podemos hacer lo mismo, pues estamos buscando costo $O(1)$ amortizado y subir un nodo a la raíz cuesta $O(\log n)$. La idea entonces es forzar el costo constante *cortando* el nodo de su padre directamente para luego agregarlo a la lista de árboles de la cola. Para esto, se debe marcar los nodos que hayan perdido un hijo desde la última vez que pasaron a ser raíz.

Cortar sin control degradaría los árboles, así que se aplica la regla de los **cortes en cascada**: un nodo que pierde su **segundo** hijo también es cortado y llevado a la lista de árboles, y esto se propaga hacia arriba.

En pseudocódigo:

DECREASEKEY-FIBONACCI(Q, x, c)

```
1  $x.key \leftarrow c$ 
2  $y \leftarrow x.parent$ 
3 if  $y \neq \text{null}$  and  $x.key < y.key$ :
4    $\text{cut}(Q, x, y)$ 
5    $\text{cascadingCut}(Q, y)$ 
6 Actualizar el mínimo de  $Q$  si corresponde
```

CUT(Q, x, y)

- 1 Sacar a x de la lista de hijos de y y decrementar $y.degree$
 - 2 Agregar x a la lista de árboles de Q
 - 3 $x.parent \leftarrow \text{null}, x.flag \leftarrow \text{False}$
-

CASCADINGCUT(Q, y)

- 1 $z \leftarrow y.parent$
 - 2 **if** $z \neq \text{null}$:
 - 3 **if** $y.flag = \text{False}$:
 - 4 $y.flag \leftarrow \text{True}$
 - 5 **else**:
 - 6 $\text{cut}(Q, y, z)$
 - 7 $\text{cascadingCut}(Q, z)$
-

Pregunta: Demuestren que el costo amortizado de **decreaseKey** en una cola de Fibonacci es $O(1)$, extendiendo el análisis de la sección 2.5.2 del apunte. No olviden mostrar que la nueva función potencial utilizada es válida para el análisis.

Hint: Agreguen un tercer término proporcional a la cantidad de nodos marcados a la función potencial ya definida y luego analicen una llamada con c cortes en cascada.

3.4. Construcción inicial de Q

Recuerden que, en estas dos colas, la construcción de Q a partir de los $|V|$ costos iniciales (*heapify*) cuesta $O(|V|)$ y se realiza **mediante inserciones sucesivas** (secciones 2.4.3 y 2.5.1 del apunte). Argumente en su informe por qué ambas estructuras cumplen este costo.

4. El algoritmo de Prim

El algoritmo de Prim construye el MST de forma incremental: parte desde un nodo raíz arbitrario r y, en cada paso, agrega la arista de menor peso que conecta un nodo ya incluido en el árbol con uno que aún no lo está. La cola de prioridad mantiene, para cada nodo fuera del árbol, el peso de la arista más barata que lo conecta con el árbol actual.

4.1. Pseudocódigo

PRIM(G, r)

- 1 $\text{costos}[r] \leftarrow 0, \text{parent}[r] \leftarrow -1$
 - 2 **for** $v \in V \setminus \{r\}$:
-

PRIM(G, r)

```
3  | costos[v] ← ∞, parent[v] ← indefinido
4  Q ← construir(costos)
5  T ← ∅
6  while Q ≠ ∅:
7    (c, v) ← extractMin(Q)
8    if v ≠ r: T ← T ∪ {(parent[v], v)}
9    for u ∈ vecinos(v) tal que u ∈ Q:
10   | if w(v, u) < costos[u]:
11   |   costos[u] ← w(v, u)
12   |   parent[u] ← v
13   |   decreaseKey(Q, u, w(v, u))
14 return T
```

4.2. Algoritmo 1: Prim con colas binomiales

Como estructura Q se utiliza una **cola binomial**, que soporta **extractMin** y **decreaseKey** en tiempo $O(\log v)$ en el peor caso. Dado que se realizan v llamadas a **extractMin** y hasta e llamadas a **decreaseKey**, la complejidad resultante del algoritmo es $O(e \log v)$. Realicen el análisis de la complejidad del algoritmo e inclúyanla en su informe.

4.3. Algoritmo 2: Prim con colas de Fibonacci

Como estructura Q se utiliza una **cola de Fibonacci**, que implementa **extractMin** en $O(\log v)$ amortizado y **decreaseKey** en tiempo constante **amortizado**. La complejidad resultante del algoritmo es $O(e + v \log v)$. Realicen el análisis de la complejidad del algoritmo e inclúyanla en su informe.

5. Implementación

Tanto la estructura del grafo como ambas colas de prioridad deben ser implementadas **en su totalidad** por ustedes. No se permite el uso de librerías que resuelvan directamente el problema (por ejemplo, `std::priority_queue` en C++, `PriorityQueue` en Java o cualquier grafo prefabricado).

Para esta tarea deberán programar entonces:

- El generador de grafos aleatorios conexos mediante listas de adyacencia para fines de reproducibilidad de sus propios experimentos. Pueden almacenarlas como un archivo de texto.
- El algoritmo de Prim_{MST} usando una cola binomial.
- El algoritmo de Prim_{MST} usando una cola de Fibonacci.

Además del código, deberán evaluar el rendimiento de ambas estructuras en términos de tiempo de ejecución total (comparando con sus cotas teóricas) y costo por operación, de modo de evidenciar la naturaleza amortizada de `decreaseKey` para las colas de Fibonacci.

Pregunta: Finalmente, deberán proponer alguna aplicación de cualquiera de sus dos implementaciones a un problema real que les parezca interesante (no es necesario programarla, solo inclúyanlo en su informe), justificando por qué creen que podría ser útil en términos de los costos de los algoritmos.

6. Experimentación

6.1. Generación de los datasets

Los grafos deben contener $v = 2^i$ nodos y $e = 2^j$ aristas con pesos $w(e)$ aleatorios en el rango $(0, 1]$. La distribución de los pesos queda a disposición de ustedes (deben documentarlo en su informe), pero el par (i, j) debe cumplir ciertos requisitos que detallaremos más adelante, pues varían entre experimentos.

Se sugiere el siguiente método para generar grafos conexos aleatorios:

- Agregar $v - 1$ aristas al grafo de tal forma que generen un árbol cobertor. Una forma de hacer esto es que para cada nodo v_i , se lo conecta a un nodo aleatorio elegido en $[1..i - 1]$.
- Luego de tener un grafo conexo, añadir las $2^j - v + 1$ aristas restantes de manera aleatoria, descartando y volviendo a sortear aquellas que resulten repetidas o reflexivas.

6.2. Consumo de memoria

Antes de ejecutar los experimentos, estimen analíticamente la memoria requerida por el caso más grande posible ($v = 2^{15}$, $e = 2^{20}$) para cada una de sus implementaciones, considerando por separado:

- La lista de adyacencia. Recuerden que cada arista se almacena **dos veces**, una por cada sentido.
- Los nodos de la cola de prioridad. Consideren que cada estructura guarda, por nodo, cierta cantidad de punteros más algunos valores adicionales. Investiguen cuanto espacio ocupa cada uno y extiendan eso al caso pedido.
- Los arreglos auxiliares `costos`, `parent` y el arreglo de punteros a los nodos de Q .

Reporten esa estimación en el informe, comparada con la memoria RAM disponible en su máquina, e indiquen si generó un problema o si tuvieron que ajustar algo para evitarlo.

¿Qué valores podrían generar problemas en su máquina? Esta pregunta es opcional y no tiene puntaje asignado, pero es interesante preguntarse estas cosas de repente. Si encuentran una respuesta, por favor inclúyanla en su informe.

6.3. Escenarios

Cada configuración (i, j) debe ejecutarse **10 veces**, con un grafo distinto en cada repetición, reportando el promedio de las mediciones en una tabla. De esta forma se puede distinguir la diferencia real entre ambas estructuras del ruido de la máquina, y obtener datos estadísticamente significativos.

6.3.1. Costo total

Ejecuten **ambos algoritmos** sobre las siguientes 10 configuraciones, midiendo el tiempo total de ejecución (la generación del grafo y la lectura de datos **no** deben incluirse en la medición):

- **Serie A (v fijo):** $i = 20$, con $j \in \{20, 21, 22, 23, 24\}$.
- **Serie B (e fijo):** $j = 24$, con $i \in \{18, 19, 20, 21, 22\}$.

Reporten los tiempos en tiempos en una tabla, incluyendo el promedio de cada configuración y gráfíquenlos en **cuatro** gráficos de línea, variando el tipo de cola de prioridad y la serie. Es decir: Cola binomial frente a serie A, cola binomial frente a serie B, cola de Fibonacci frente a serie A y cola de Fibonacci frente a serie B.

Los gráficos deben incluir la curva del algoritmo correspondiente junto a su cota teórica, multiplicada por alguna constante a determinar. Recuerden usar la misma escala en ambos gráficos de una misma serie para poder comparar efectivamente el rendimiento de ambas versiones del algoritmo frente a una misma configuración.

Finalmente, verifiquen que ambas implementaciones produzcan un MST del mismo peso total. Reporten esta verificación en el informe.

6.3.2. Costo amortizado

Ejecuten **ambos algoritmos** sobre las siguientes 10 configuraciones:

- **Serie C (v fijo):** $i = 18$, con $j \in \{18, 19, 20, 21, 22\}$.
- **Serie D (e fijo):** $j = 22$, con $i \in \{14, 15, 16, 17, 18\}$.

En esta parte, deberán medir el costo de las llamadas a `decreaseKey` de dos maneras:

- a) **Tiempo acumulado:** sumar el tiempo de todas las llamadas y graficarlo contra la cantidad de llamadas realizadas.
- b) **Conteo de operaciones:** contar las operaciones estructurales que realiza cada llamada (cortes en cascada en la cola de Fibonacci, intercambios en la binomial) en vez de medir tiempo.

Grafiquen los resultados en **ocho** gráficos de línea: Tiempo acumulado de cola binomial en serie A, tiempo acumulado de cola binomial en serie B, conteo de operaciones de cola binomial frente a serie A, conteo de operaciones de cola binomial frente a serie B, tiempo acumulado de cola de Fibonacci en serie A, tiempo acumulado de cola de Fibonacci en serie B, etc.

Los gráficos deben incluir la curva del algoritmo correspondiente junto a su cota teórica, multiplicada por alguna constante a determinar. Recuerden usar la misma escala en ambos gráficos

de una misma serie y forma de medición para poder comparar efectivamente el rendimiento de ambas versiones del algoritmo frente a una misma configuración.

Recomendación: Guarden las mediciones en alguna estructura en memoria y procésenlas **después** de terminar el algoritmo, para no interferir con el tiempo de ejecución.

Pregunta: ¿Tienen sentido las formas de sus gráficos en esta sección? Argumenten.

7. Entregables

Se deberá entregar el código fuente y un informe en formato PDF con los resultados y conclusiones obtenidas luego de la experimentación. La nota final de la tarea (NT) será el promedio simple entre la nota de código (NCod) y la nota de informe (NInf):

$$NT = \frac{1}{2} \cdot \text{NCod} + \frac{1}{2} \cdot \text{NInf}$$

7.1. Código

La entrega de código debe contener:

- **(0.5 pts) README:** Archivo con instrucciones claras para compilar y ejecutar el código. Debe ser lo suficientemente explicativo para que cualquier persona pueda ejecutar la totalidad de los experimentos.
- **(0.2 pts) Firmas:** Cada estructura de datos y función relevante debe tener una descripción de su propósito y de sus parámetros de entrada y salida.
- **(0.5 pts) Generación de grafos:** Creación correcta de los nodos y aristas según lo pedido, garantizando conectividad y simplicidad.
- **(1.5 pts) Prim con cola binomial:** Implementación correcta de la estructura y del algoritmo.
- **(1.8 pts) Prim con cola de Fibonacci:** Implementación correcta de la estructura y del algoritmo, incluyendo los cortes en cascada.
- **(1.0 pts) Obtención de resultados:** La medición de tiempos es rigurosa y suficiente para emitir conclusiones.
- **(0.5 pts) Main:** Un archivo principal que permita ejecutar toda la batería de experimentos sin modificar el código.
- **(1.0 pts extra) Bonus:** Implementación de su aplicación propuesta en la [sección 5](#). Puede ser una versión reducida de su idea, la idea es que no sea mucho más código del que ya hicieron, pero que sea capaz de entregar resultados de algún tipo.

7.2. Informe

El informe debe ser claro, analítico y conciso. Se recomienda encarecidamente utilizar LaTeX o Typst. Debe contener:

- **(0.6 pts) Introducción:** Presentación del problema del MST, resumen de lo que se abordará en el informe y planteamiento de una hipótesis sobre el desempeño esperado de cada estructura.
- **(0.8 pts) Desarrollo:** Breve explicación de las estructuras y del algoritmo. Dado que el cuerpo docente ya conoce la teoría, enfóquense en explicar **sus** decisiones de implementación: cómo resolvieron el acceso directo a los nodos de Q , cómo implementaron `decreaseKey` en cada estructura y cómo construyeron la cola inicial. No repitan la descripción de las colas que ya está en el apunte.
- **(2.0 pts) Resultados:** Especificación detallada del *setup* experimental (CPU, RAM, tamaños de caché, SO, lenguaje y versión del compilador, *flags* de compilación). Se deben mostrar los doce gráficos solicitados con sus ejes correctamente etiquetados, la tabla de tiempos con promedio, la verificación de que ambos MST pesan lo mismo y la estimación de memoria pedida.
- **(1.8 pts) Análisis:** Discusión de los resultados obtenidos e inclusión de las respuestas a las tres preguntas planteadas en las secciones [3.3](#), [5](#) y [6.3.2](#). ¿Se refleja la diferencia entre $O(e \log v)$ y $O(e + v \log v)$ en los tiempos reales? ¿Qué constantes necesitaron para ajustar las cotas teóricas a sus mediciones, y qué dice eso sobre el costo real de cada estructura? Si las curvas no siguen a sus cotas, ¿qué factores fuera del modelo teórico pueden explicarlo?
- **(0.8 pts) Conclusión:** Recapitulación del trabajo, verificación de la hipótesis planteada en la introducción y mención de posibles mejoras o trabajo futuro.

8. Recomendaciones para la tarea

- a) Implementen primero la cola binomial: la unión de dos árboles B_k es la primitiva que después reutiliza el `extractMin` de la cola de Fibonacci, así que llegarán a la segunda estructura con la mitad del trabajo hecho.
- b) Verifiquen sus estructuras con casos pequeños antes de correr los experimentos grandes: un grafo de ≈ 10 nodos dibujado a mano permite comparar el MST obtenido contra la respuesta correcta.
- c) Que ambos algoritmos entreguen el mismo peso total es una excelente prueba de regresión, pero recuerden que el MST **no** es necesariamente único si hay pesos repetidos.
- d) Cuiden que la medición de tiempos use un reloj monótono de alta resolución (`clock_gettime`, `std::chrono::steady_clock`, `System.nanoTime`) y no el tiempo de pared del sistema.
- e) Por favor no incluyan sus datasets en sus entregas 🙏.